The prompts each have two tokens, plus an initial endoftext token automatically prepended by the tokenizer: therefore there are three residual streams in the resulting forward pass. For each residual stream $s^{(i)}$, we plot a line showing the L_2 norm of the steering vector at that sequence position (e.g. the Ang-Cal activations at position 1), divided by the norm of the residual stream at that position (i.e. the prompt embedding, here 'I' at position 1).

$$\text{RelativeNorm}_{h_A}(i) = \frac{||h_A^{(i)}||}{||s^{(i)}||}$$

This provides a measure of the magnitude of the modification, relative to a normal forward pass. Figure 9 shows the resulting relative norm over layer number.

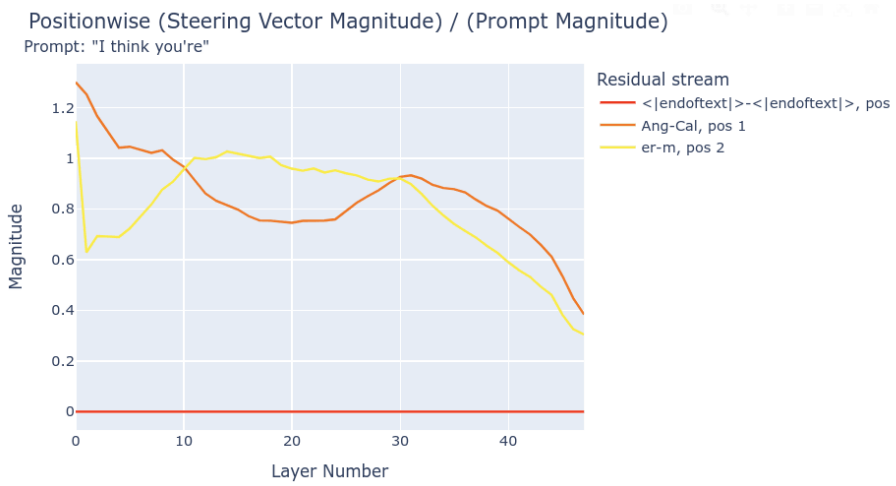

Figure 9: The relative norm decreases throughout the forward pass. The flat red line is because position 0 is the same token (`endoftext`) for both 'Anger' and 'Calm', and so the difference is 0. Thus, position 0 is never modified by a steering vector generated from any pair of prompts.

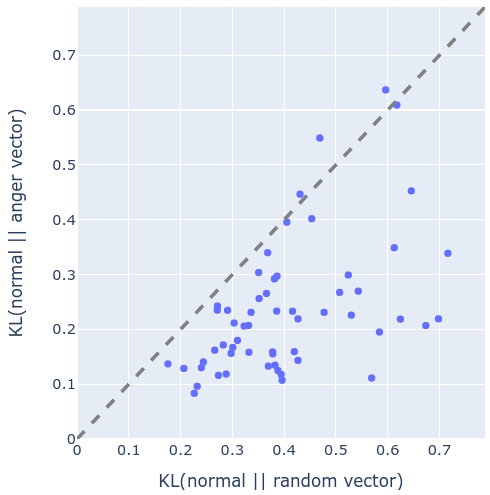

Figure 10: The KL-divergence of output tokens under an `anger` ActAdd and under a random vector. We see that, systematically, the anger vector changes the output distribution less than a random vector.

Importantly, Figure 9 shows the result of using $c = +1$. But `Anger` $-$ `Calm` is an effective steering vector at coefficient $+10$. Therefore, this intervention is nearly ten times the norm of the underlying forward pass. Heuristically, we interpret this as meaning that after layer normalization (and ignoring

any destructive interference from adding the steering vector), around 90% of the residual stream is determined by the steering vector and not by the previous information computed from the prompt ("I think you're"). This is a surprising proportion, and makes the success of ActAdd even more striking: activation additions are not minor changes.

G INVESTIGATING RANDOM ACTADD VECTORS

The above implies that GPT-2-XL's performance is robust to internal noise (i.e. bad activations or destructive parts of steering vectors). We test this by injecting random vectors with similar magnitudes to the steering vectors.

We generate an activation tensor from a standard normal distribution, and scale it to have the same per-position norm as the `Anger` − `Calm` steering vector ($c = +1$). We then inject it into the forward pass at the appropriate location. Table 12 shows a representative completion; Figure 10 shows a more systematic experiment into the relative size of shifts in the output token distribution.

The random vector seems not to modify the qualitative distribution of completions. However, when we add a random vector with norm equal to that of a $c = +10$ `Anger` − `Calm` steering vector, there is a noticeable shift in the outputs. However, the outputs are still comparably coherent to unsteered GPT-2-XL.

This is evidence that GPT-2-XL is somewhat resistant to random perturbation, and is instead controllable through consistent feature directions which are added to its forward pass by steering vectors.

We quantitatively support this conclusion by testing how each modification changes the model's probability distribution over next tokens. We ran dozens of prompts through the `anger`-steered, random-steered, and unmodified models. Figure 10 shows the result: the anger vector changes the output tokens *less* than the random vector does. This suggests that the anger vector has more targeted effects on next-token probabilities.

Note that random vectors are not the same as the steering vectors for random (i.e. character-level uniformly distributed) text. We thus also tried the '`fdsajl; fs`' − (whitespace) vector. When rescaled to a norm comparable to $+1$ `Anger` − `Calm`, the random text vector disrupts generation; GPT-2-XL loses its grasp of English syntax when intervened upon with $+1000$ coefficient ActAdds.

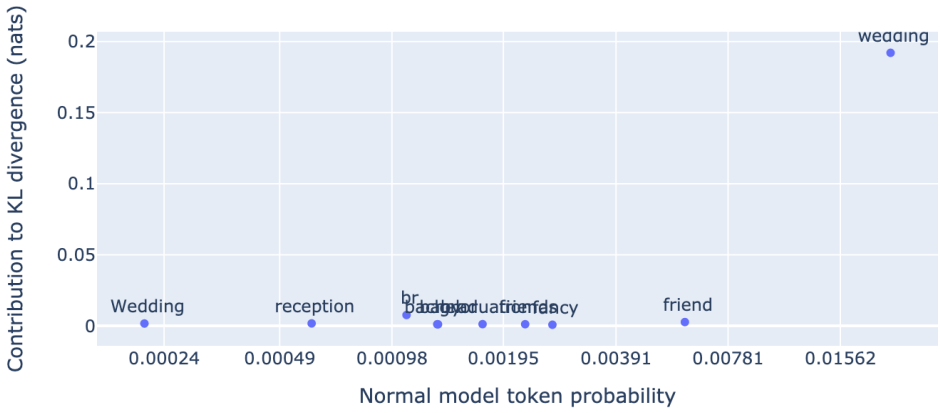

Figure 11: Token-level effect of the ActAdd wedding vector on KL-divergence, using GPT-J-6B instead of GPT-2.

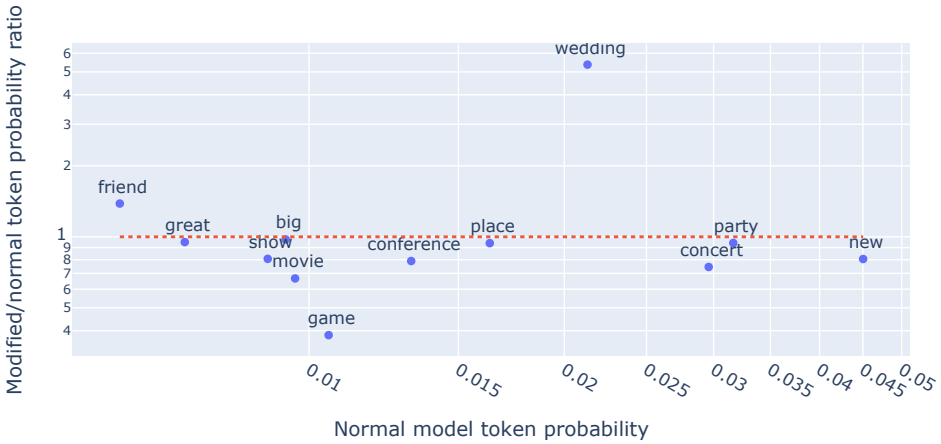

Figure 12: Token-level effect of the ActAdd wedding vector on token probability, using GPT-J-6B instead of GPT-2.

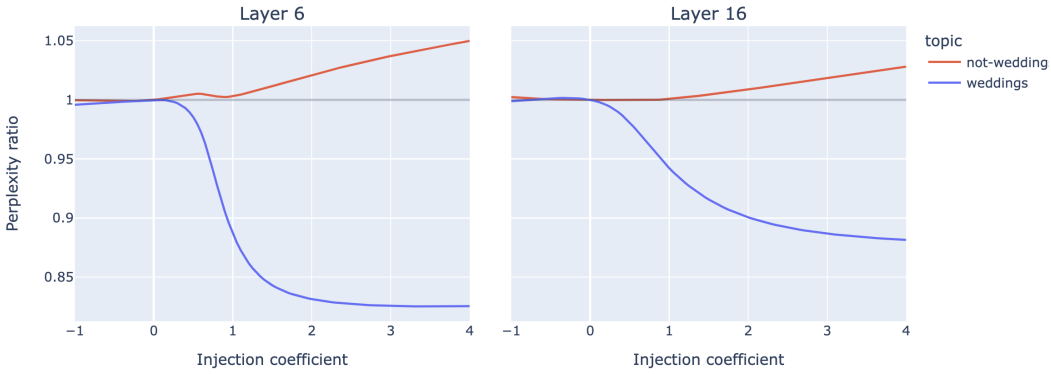

Figure 13: Perplexity ratio effect of the ActAdd wedding vector (blue) across different steering coefficient values, using GPT-J-6B instead of GPT-2. (L) when injecting the steering vector at layer 6; (R) when at layer 16.

H Partial ActAdd

GPT-2-XL has a 1600-dimensional residual stream. Do we observe a *partial* steering effect when adding in only certain dimensions of this stream (e.g., dimensions 0 through 799)? Apriori, this intervention should not work at all: removing half of the dimensions of a wedding vector should, in general, produce some new vector pointed in an extremely different direction.

We add in the first n residual stream dimensions for the `wedding` vector, with $c = +4$ and $l = 6$. For a range of fractions of total dimensions $f \in [0/1600, 160/1600, ..., 1600/1600]$ and for each of six prompts p_i, we generated 100 completions. For each f and p_i, we plotted the average number of wedding words per completion. (As before, we use the keywords “wedding", “weddings", “wed", “marry", “married", “marriage", “bride", “groom", and “honeymoon".)

Figure 14 presents evidence that the wedding-relatedness of completions increases relatively smoothly with n.

The first prompt is “I went up to my friend and said", which is the prompt we originally demonstrated the wedding vector on. For this prompt, we observe a non-monotonic relationship between weddingness and fraction of dimensions modified. Surprisingly, for the first prompt, adding in the first 1,120 dimensions of the residual stream makes the completions more about weddings than all 1,600 dimensions. We originally chose this prompt to give GPT-2 an opportunity to bring up weddings.